KUBERNETES , CONTENEDORES, MICROSERVICIOS EN LA NUBE

J. S. Morales López

7690-08-6790 Universidad Mariano Gálvez

Seminario de tecnologías de información

jmorales115@miumg.edu.gt

Repositorio en GitHub - Sofia Morales

Resumen

El presente artículo aborda la orquestación de servidores y contenedores, así como los conceptos fundamentales para implementar aplicaciones basadas en microservicios dentro de entornos de nube. Su objetivo es analizar la relación existente entre Docker, Kubernetes, OAuth 2.0 y la metodología de los Doce Factores. Docker permite empaquetar cada microservicio junto con sus dependencias dentro de un contenedor. Para ello, el Dockerfile contiene las instrucciones necesarias para construir la imagen, mientras que Docker Compose facilita la definición y ejecución de aplicaciones compuestas por varios servicios. Kubernetes, por su parte, permite administrar los contenedores mediante elementos como los Pods, Deployments y Services. Estos componentes facilitan la comunicación interna, el mantenimiento de réplicas y el escalado de las aplicaciones. Asimismo, Minikube permite crear un entorno local para aprender y probar el funcionamiento de Kubernetes, mientras que kubectl proporciona los comandos para administrar el clúster. En materia de seguridad, OAuth 2.0 permite controlar el acceso a los recursos mediante tokens de autorización. Como resultado del análisis, se identifica que estas tecnologías, complementadas con los principios de los Doce Factores, favorecen el desarrollo de aplicaciones escalables, disponibles, seguras y preparadas para su implementación en la nube.

Palabras clave: Docker, microservicios, Kubernetes, OAuth2.0, .

Desarrollo del tema

La orquestación de servidores y contenedores permite coordinar los componentes que forman una aplicación y administrar su ejecución de manera automatizada. Esta práctica resulta especialmente importante en aplicaciones basadas en microservicios, debido a que pueden estar compuestas por numerosos servicios que deben comunicarse, escalarse y mantenerse disponibles dentro de una infraestructura local o en la nube.

Microservicios y contenedores

La arquitectura de microservicios divide una aplicación en servicios pequeños que representan diferentes funciones del negocio. Cada microservicio puede desarrollarse, desplegarse y escalarse de manera independiente. También puede emplear un lenguaje de programación o un mecanismo de almacenamiento diferente, siempre que mantenga una comunicación claramente definida con los demás servicios.

Esta separación puede facilitar la continuidad operativa porque una falla puede aislarse en un compo-

nente determinado. Sin embargo, los microservicios no garantizan por sí solos la alta disponibilidad, ya que algunos servicios pueden depender de otros para completar una operación. Por esta razón, se necesitan mecanismos de monitoreo, recuperación, balanceo y tolerancia a fallos.

Docker permite empaquetar un microservicio junto con sus dependencias dentro de una imagen. El Dockerfile contiene las instrucciones necesarias para construir dicha imagen, mientras que Docker Compose permite definir y ejecutar una aplicación formada por varios contenedores. En un archivo Compose pueden configurarse los servicios, las redes, los volúmenes y otras condiciones necesarias para su ejecución.

Orquestación mediante Kubernetes

Cuando la cantidad de contenedores aumenta, su administración manual se vuelve más compleja. Kubernetes permite orquestar aplicaciones contenerizadas, controlando su despliegue, comunicación, escalamiento y recuperación. Los contenedores se ejecutan dentro de Pods, que constituyen las unidades desplegables más pequeñas administradas por Kubernetes.

Un Deployment declara el estado deseado de una aplicación, incluyendo la cantidad de réplicas que deben permanecer activas. Si un Pod presenta una falla, Kubernetes puede crear otro para mantener el número solicitado. Los Services proporcionan un punto estable de comunicación entre los componentes, aunque los Pods sean reemplazados y cambien de dirección dentro del clúster.

La herramienta kubectl permite administrar Kubernetes mediante comandos, mientras que Minikube facilita la creación de un clúster local para realizar pruebas y conocer sus componentes. En ambientes productivos, Kubernetes puede implementarse sobre servidores propios o mediante servicios administrados de proveedores de nube.

Autorización mediante OAuth 2.0

La comunicación entre microservicios también requiere controles de seguridad. OAuth 2.0 es un marco de autorización que permite conceder acceso limitado a recursos protegidos mediante tokens. Los permisos pueden delimitarse mediante alcances o scopes, evitando que una aplicación obtenga más acceso del necesario.

OAuth 2.0 no debe describirse directamente como un mecanismo de autenticación. Cuando se necesita verificar la identidad del usuario e implementar un inicio de sesión, puede utilizarse OpenID Connect sobre OAuth 2.0. Su implementación debe proteger los tokens, limitar sus permisos y utilizar conexiones cifradas para reducir el riesgo de accesos no autorizados.

Implementación en la nube

Metodología de los Doce Factores

Observaciones y comentarios

Conclusiones

1. La arquitectura debe seleccionarse tomando en cuenta los requisitos funcionales, los atributos de calidad y las restricciones del proyecto.

2. Una aplicación pequeña puede utilizar una arquitectura monolítica o en capas sin asumir innecesariamente la complejidad de un sistema distribuido.

3. No se debe asumir la complejidad de una aplicación sin realizar previamente un análisis de sus usuarios, funciones, presupuesto y crecimiento esperado.

4. Una arquitectura de microservicios puede ser apropiada para aplicaciones complejas que requieren escalabilidad e independencia entre sus servicios.

5. La experiencia del equipo, el presupuesto y las necesidades de mantenimiento también deben evaluarse para seleccionar una arquitectura adecuada para la organización.

Referencias

https://docs.docker.com/ https://www.databricks.com/es/blog/what-is-orchestration https://www.advsyscon orchestration-what-is/ https://kubernetes.io/es/ https://cloud.google.com/learn/what-is-kubernetes?hl=es https://www.ibm.com/mx-es/think/topics/kubernetes https://aws.amazon.com/es/microservices/ https://cloud.google.com/learn/what-is-microservices-architecture?hl=es-419 https://www.ibm.com/mx-es/think/topics/microservices https://oauth.net/2/ https://auth0.com/es/intro-to-iam/what-is-oauth-2 https://medium.com/@csw11235/understanding-the-12-factor-app-methodology-why-we-build-cloud-applications-the-way-we-do-7aed4a98fbd8 https://medium.com/cloud-native-daily/what-are-12-factor-principles-of-cloud-native-microservices-eb25e95e5906

Fowler, M. (2015, 1 de julio). *Microservice trade-offs*. MartinFowler.com. https://martinfowler.com/articles/microservice-trade-offs.html

Kazman, R., Echeverria, S., & Ivers, J. (2023). *A holistic view of architecture definition, evolution, and analysis* (CMU/SEI-2023-TR-004). Software Engineering Institute, Carnegie Mellon University. https://doi.org/10.1184/R1/20722402

Microsoft. (2025, 14 de octubre). *Architecture styles*. Azure Architecture Center. https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/

Saraswathi, R. (s. f.). *Tipos de arquitecturas de desarrollo de aplicaciones*. IBM. https://www.ibm.com/es-es/think/topics/application-architecture-types